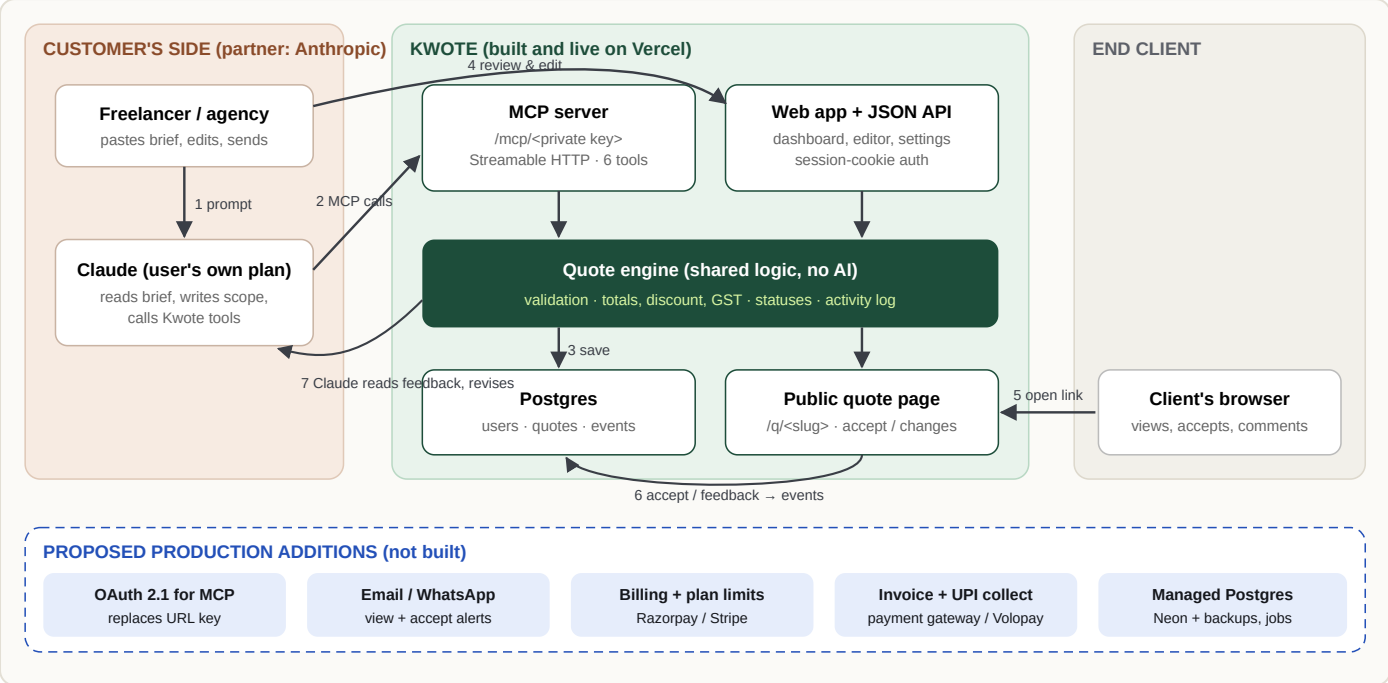
Architecture diagram



Solid boxes and arrows are built and live. The dashed band is the proposed production system. Numbers show the order of the data flow. No AI model runs inside Kwote: all reasoning and writing happens in the customer's own Claude.

DATA FLOW

1. **Prompt:** the freelancer pastes a client brief into their own Claude, with the Kwote connector turned on.
2. **MCP calls:** Claude calls `get_business_profile` to read the rate card, currency, GST and terms. It writes the quote, then calls `create_quote` with structured fields (title, client, summary, line items, timeline, terms).
3. **Save:** the quote engine validates the input, calculates subtotal, discount, tax and total in code, then stores the quote and a "created" event in Postgres. It returns an edit link and a client link to Claude.
4. **Review and edit:** the dashboard polls every 4 seconds, so the new quote appears without a refresh. The owner edits fields in the editor, which autosaves through the JSON API. The same engine recalculates totals and logs an "edited" event.
5. **Share:** the owner sends the client link. Visits by anyone other than the owner count as views, and the status moves from draft or sent to viewed.
6. **Respond:** the client accepts with a typed name, or requests changes. The status and events update, and the dashboard shows the change.
7. **Revise:** Claude calls `list_quotes` and `get_quote` to read the feedback, then `update_quote` to revise. The owner can edit again.

Kwote

BUILT VS PROPOSED

Area	BUILT · live	PROPOSED · production
Claude integration	Remote MCP server, stateless Streamable HTTP, 6 tools. A private key per account in the URL, which can be reset.	OAuth 2.1 for the connector, per-tool scopes and an audit log. Listing in the Claude connector directory.
Quotes	Create and edit from Claude or the app. Line items, discount, GST, currency, timeline, terms and validity. Totals calculated in code.	Templates, quote versioning and diffs, multi-currency rates, e-signature, GST invoice numbering.
Client experience	Public branded page: accept with typed name, request changes, print to PDF. View counting.	Custom domain and branding, a server-generated PDF, reminders and expiry nudges.
Notifications	In-app only: status, activity log and live dashboard.	Email and WhatsApp alerts on views, acceptance and feedback.
Money	Pricing shown on the landing page only. No billing.	Subscriptions and plan limits. Quote → invoice → UPI and card collection.
Data	Postgres schema (users, quotes, events). On the live demo it runs as an embedded Postgres (PGlite) inside the server.	Managed Postgres (Neon, which the code already supports through `DATABASE_URL`), backups, and a background job queue.
Accounts	Email and password sign-up (scrypt hashing), HMAC-signed session cookie.	Google sign-in, teams and roles, SSO for agencies.

VOLOPAY VS PARTNER RESPONSIBILITIES (PROPOSED PRODUCTION)

Owner	Responsible for
Volopay / Kwote team	Product and UX, the quote engine and data model, the MCP server and its tool design, accounts, security and access control, pricing and billing logic, customer support, and the roadmap from quotes to invoices and payments, building on Volopay's SMB finance expertise.
Anthropic (Claude)	All AI reasoning and writing, the MCP client and connector UI, and tool-permission prompts. Paid for by the customer's own Claude plan.
Vercel	Hosting, serverless functions, TLS, CDN and deploys.
Neon (Postgres)	Managed database, backups and point-in-time restore.
Payment gateway (Razorpay, Stripe, or Volopay rails)	Subscription billing, and later invoice collection through UPI, cards and bank transfer, plus compliance (KYC, PCI).
Messaging (Resend or WhatsApp Business API)	Delivering view, acceptance and reminder notifications.

TOOLS USED

- **Runtime:** Node.js, Express 5
- **MCP:** `@modelcontextprotocol/sdk` with Zod schemas
- **Data:** Postgres through `pg`, with PGlite (embedded Postgres) as a fallback
- **Frontend:** server-rendered HTML and vanilla JS. No framework, no build step.
- **Hosting:** Vercel, deployed directly through the Vercel API.
- **Testing:** an MCP SDK client script, curl API tests, and Playwright screenshots
- **Development:** Claude was used to help write the code. There is no AI inside the app.

ASSUMPTIONS

- Target users already pay for Claude, or will, and are comfortable adding a custom connector.
- Indian freelancers will pay about ₹499 a month for faster quotes and acceptance tracking.
- Defaults are INR with 18% GST. Each account can change currency and tax.
- Cost estimates (about ₹15 per customer per month) are approximate.

LIMITATIONS (CURRENT PROTOTYPE)

- The live demo uses an embedded database inside the server, so a server restart can reset data to the sample state.
- MCP access works through a secret URL key, not OAuth. Anyone with the link can write to that account.
- No email or WhatsApp notifications, no billing, no server-side PDF.
- Acceptance is a typed name, not a legally verified e-signature.
- The dashboard updates by polling every 4 seconds rather than with websockets.
- The demo account's sample quotes and their history are seeded data.

NEXT STEPS

1. Connect Neon Postgres and add backups (one environment variable).
2. Add OAuth to the MCP connector and submit it to the Claude directory.
3. Email and WhatsApp alerts for views and acceptance, plus follow-up reminders.
4. Billing with plan limits, and custom branding for Pro.
5. Quote → GST invoice → UPI collection, then measure time-to-acceptance and win rate.
6. Pilot with 20 freelancers. Track activation as the first quote accepted within 7 days.